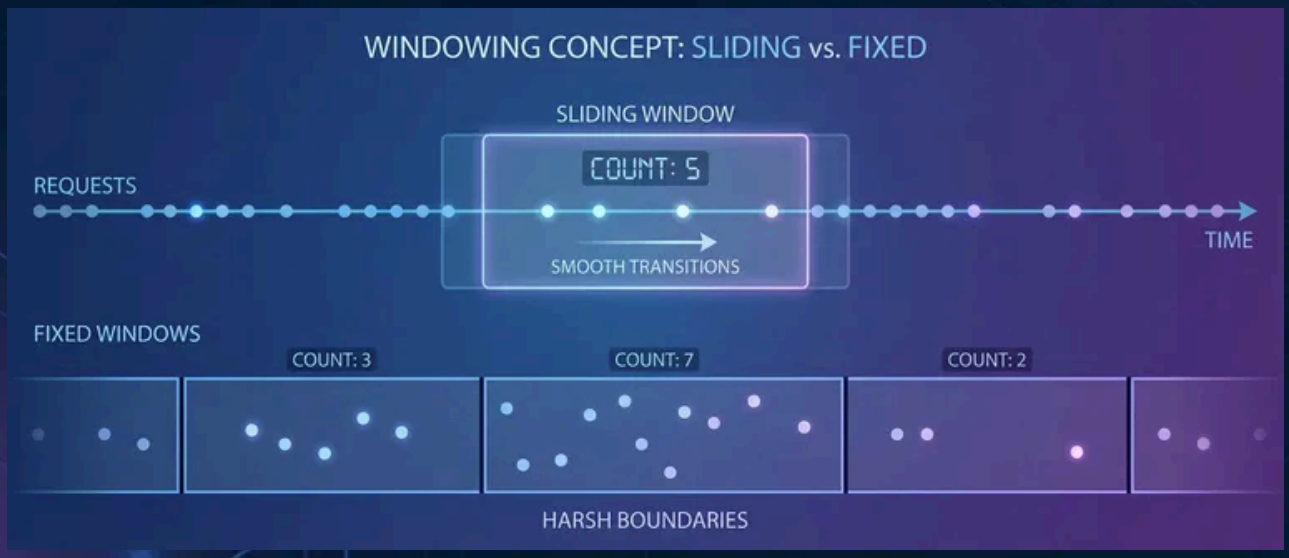
Rate Limiting Done Right: Protecting Users From Yourself



Published on January 18, 2026

Webstack Builders

Table of Contents

Rate limiting is a double-edged sword. Done right, it protects your systems from overload and abuse. Done wrong, it becomes a self-inflicted outage — your own infrastructure rejecting legitimate users during the moments you need capacity most.

I watched this play out at a company that implemented per-IP rate limiting at 100 requests per minute to prevent scraping. Seemed reasonable. Then a customer behind corporate NAT reported they couldn't use the service. Five hundred employees sharing one public IP meant each person got 0.2 requests per minute. They switched to API key-based limiting with per-key quotas. Problem solved — until a viral moment hit and legitimate traffic spiked 10x. Their fixed-window rate limiting rejected 90% of requests at minute boundaries. Users hammering refresh made it worse. They switched to a **sliding window counter** to track request counts (eliminating the boundary problem) combined with **token bucket** for burst control (allowing legitimate traffic spikes while maintaining sustained rate limits). Same traffic spike: requests distributed smoothly, everyone got served, the backend hummed along at capacity without falling over.

The naive approach—"block anything over N requests"—fails because it treats rate limiting as a wall instead of a valve. The algorithms matter: token bucket, leaky bucket, sliding window, and fixed window each behave differently during bursts. The implementation matters more: *where* you limit, *how* you identify clients, *what* happens when limits are hit, and *how* you communicate constraints to callers.

Rate limiting isn't about saying "no." It's about saying "not yet" in a way that maintains service quality for everyone.

> ⚠ **WARNING**
>
> Rate limits that work fine under normal load often become the bottleneck during incidents. Design for burst handling, not steady-state. Your rate limiter should protect services, not prevent recovery.

Rate Limiting Fundamentals

Why Rate Limit

Rate limiting serves four distinct purposes, and conflating them leads to misconfigured systems.

- **Resource protection**
  Prevents individual clients from monopolizing shared resources. Without limits, one customer's bulk export can exhaust your database connection pool, causing errors for everyone else. CPU-intensive endpoints, memory-heavy operations, and database-bound queries all need protection from unbounded consumption.
- **Fairness**
  Ensures equitable access across clients. Large customers shouldn't crowd out small ones. Free-tier users shouldn't impact paying customers. Rate limits enforce the allocation you've decided is appropriate for each tier.
- **Abuse prevention**
  Stops malicious or buggy clients. Credential stuffing attacks, scraping attempts, and runaway retry loops all look like excessive request volume. Rate limiting slows these down enough to make attacks expensive and gives you time to respond.
- **Cost control**
  Caps expensive operations. Third-party API calls, ML inference requests, and storage operations have real costs. Without limits, a compromised API key or buggy client can generate surprise bills within hours.

But rate limiting has limits. It *slows* attacks; it doesn't prevent them. Determined attackers distribute across IPs and rotate credentials. Rate limiting buys time — authentication, authorization, and WAF (Web Application Firewall) rules provide actual security. Similarly, rate limiting *sheds* load; it doesn't handle it. You still need scaling for legitimate traffic. And rate limiting *helps* availability; it doesn't guarantee it. Backend failures still cause errors regardless of how well you've throttled incoming requests.

Algorithm Overview

Four algorithms dominate production rate limiting, each with different tradeoffs:

#	Algorithm	Mechanism	Burst Handling	Memory	Best For
1	Fixed Window	Count requests in time windows, reset at boundary	Poor (2x burst at boundaries)	Low (one counter per key)	Simple quotas
2	Sliding Window Log	Store timestamp of every request	Excellent	High (all timestamps)	Accuracy-critical, low volume

#	Algorithm	Mechanism	Burst Handling	Memory	Best For
3	Sliding Window Counter	Weighted average of current and previous window	Good	Low (two counters per key)	Balance of accuracy and efficiency
4	Token Bucket	Tokens added at fixed rate, consumed per request	Excellent (configurable burst)	Low	API rate limiting
5	Leaky Bucket	Requests queue, processed at constant rate	Smooths all traffic	Medium (queue)	Traffic shaping

Rate limiting algorithm comparison.

1. **The fixed window approach**
This is simple: count requests in minute-long (or hour-long) windows, reject when the count exceeds the limit, reset at the boundary. The problem is boundary bursts. A client can make 95 requests at 11:59:59 and 95 more at 12:00:01 - 190 requests in two seconds while staying under a "100 per minute" limit.

2. **Sliding window counter**
Fixes this by weighting the previous window's count. At 30 seconds into the current window, you count 50% of the previous window plus 100% of the current window. This smooths the boundary problem with minimal additional state.

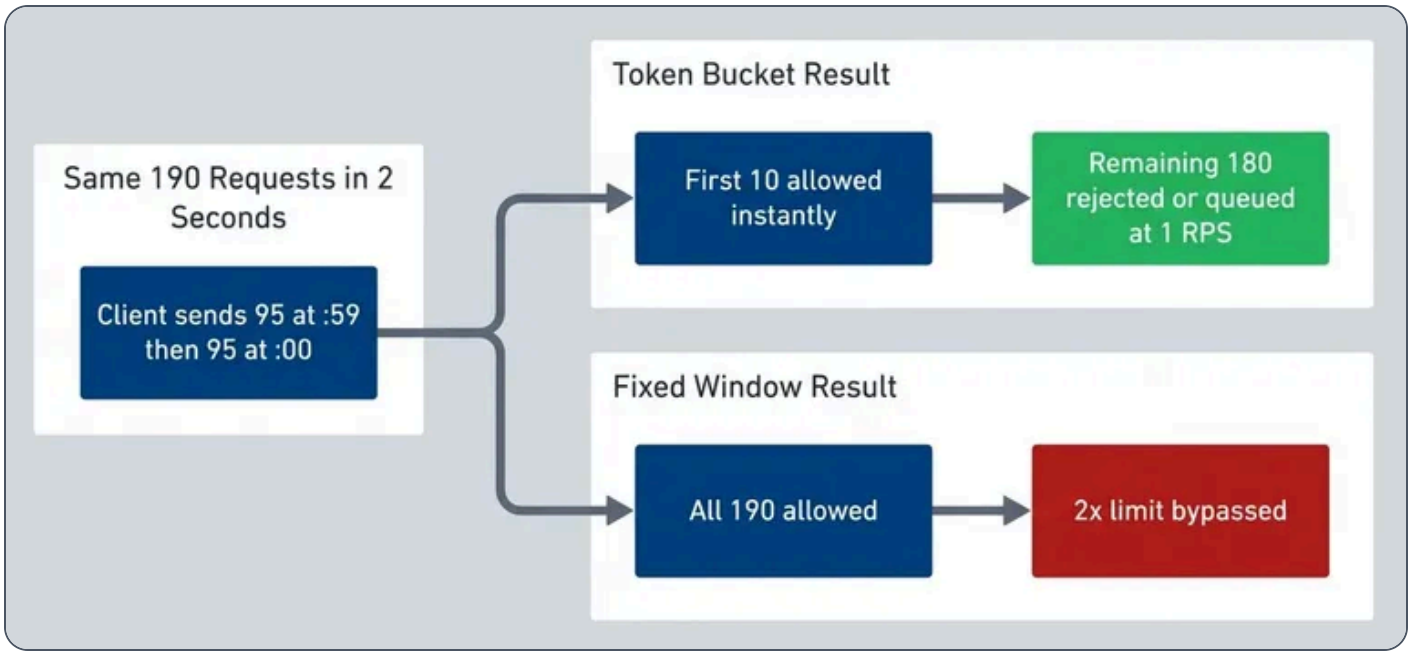
3. **Sliding window log**
Takes accuracy further by storing the timestamp of every request and counting only those within the rolling window. It's perfectly accurate but memory-intensive — you're storing potentially thousands of timestamps per client. Use it only when accuracy is critical and request volume is low, like authentication rate limiting where you absolutely need exactly N attempts per hour.

4 **Token bucket**

The workhorse of API rate limiting. Imagine a bucket that holds N tokens (your burst capacity). Tokens are added at rate R (your sustained RPS). Each request consumes a token if available; otherwise it's rejected. This naturally allows bursts — a client can use their full bucket immediately — while maintaining a sustained rate over time.

5 **Leaky bucket**

Inverts the model: requests enter a queue (the bucket), and they're processed (leaked) at a constant rate. If the queue fills, new requests overflow and are rejected. This produces perfectly smooth output but adds latency — requests wait in the queue. Use it when you need constant output rate to a downstream service that can't handle bursts.



Same burst traffic, different outcomes.

> **INFO**
>
> Token bucket is the most versatile algorithm for API rate limiting. It allows bursts (good for user experience) while maintaining a sustained rate (protects backend). Most production rate limiters — including AWS API Gateway, Kong, and nginx — use token bucket or sliding window.

Where to Rate Limit

Before choosing an algorithm, decide *where* in your stack to enforce limits. Each layer has different tradeoffs:

Layer	Latency	Tools	Granularity	Best For
Edge/CDN	Lowest	Cloudflare, CloudFront, Fastly	IP, geography	DDoS, bot protection
API Gateway	Low	AWS API Gateway, Kong, nginx	API key, route	API quotas, tiered plans
Service Mesh	Low	Istio, Envoy, Linkerd	Service identity	Service-to-service limits
Application	Higher	Custom middleware	User, action, context	Fine-grained business logic

Rate limiting layers comparison.

Edge and CDN

Edge rate limiting stops bad traffic before it reaches your infrastructure. Cloudflare, Fastly, AWS CloudFront, Google Cloud CDN, Azure Front Door, and Azure CDN all offer rate limiting rules via their respective WAF (Web Application Firewall) products that execute at the edge — milliseconds from the client, before your servers see the request.

cloudflare-rate-limit.tf

```
# Limit API endpoints to 100 requests per minute per IP
resource "cloudflare_rate_limit" "api_limit" {
  zone_id   = var.zone_id
  threshold = 100
  period    = 60

  match {
    request {
      url_pattern = "api.example.com/*"
      schemes     = ["HTTPS"]
    }
  }

  action {
    mode    = "simulate"  # Change to "ban" in production
    timeout = 60
  }
}
```

Cloudflare rate limiting via Terraform.

Edge limiting is coarse-grained — typically by IP or geographic region. It's your first line of defense against volumetric attacks, not your primary quota enforcement.

API Gateway

Most teams implement their primary rate limiting at the API gateway layer. AWS API Gateway, Kong, and nginx all support token bucket or similar algorithms out of the box.

kong-rate-limiting.yaml

```
# Kong rate limiting plugin configuration
plugins:
  - name: rate-limiting
    config:
      minute: 100
      hour: 1000
```

```
        policy: redis
        redis_host: redis.internal
        redis_port: 6379
        fault_tolerant: true
        hide_client_headers: false
        limit_by: consumer  # or: ip, credential, header
```

Kong rate limiting plugin configuration.

aws-api-gateway.yaml

```
# AWS API Gateway usage plan (via CloudFormation)
UsagePlan:
  Type: AWS::ApiGateway::UsagePlan
  Properties:
    UsagePlanName: BasicTier
    Throttle:
      BurstLimit: 100    # Token bucket capacity
      RateLimit: 10      # Tokens per second
    Quota:
      Limit: 10000
      Period: MONTH
```

AWS API Gateway usage plan with token bucket throttling.

Gateway-level limiting handles per-API-key quotas, tiered rate limits, and route-specific throttling without touching application code.

Service Mesh

For service-to-service communication, service meshes like Istio provide rate limiting between internal services. This prevents one service from overwhelming another during failures or traffic spikes.

istio-rate-limit.yaml

```
# Istio EnvoyFilter for local rate limiting
apiVersion: networking.istio.io/v1alpha3
kind: EnvoyFilter
```

```
  metadata:
    name: payment-service-ratelimit
  spec:
    workloadSelector:
      labels:
        app: payment-service
    configPatches:
      - applyTo: HTTP_FILTER
        match:
          context: SIDECAR_INBOUND
        patch:
          operation: INSERT_BEFORE
          value:
            name: envoy.filters.http.local_ratelimit
            typed_config:
              "@type":
type.googleapis.com/envoy.extensions.filters.http.local_ratelimit.v3.LocalRateLimit
              stat_prefix: http_local_rate_limiter
              token_bucket:
                max_tokens: 100
                tokens_per_fill: 10
                fill_interval: 1s
```

Istio local rate limiting with token bucket.

Application Layer

Application-level rate limiting gives you the finest control — you can limit by user, by action, by context, or by any combination. Use it when gateway-level limits aren't granular enough.

The tradeoff is complexity. You're responsible for state management (usually Redis for distributed systems), failure handling, and the actual algorithm implementation. That said, sometimes you need it: per-user action limits, variable-cost operations, or business-logic-driven throttling that gateways can't express.

> **⚠ WARNING**
>
> Don't implement rate limiting **only** at the application layer. By the time requests reach your app, they've already consumed network bandwidth, TLS handshakes, and load balancer capacity. Use defense in depth: coarse limits at the edge, refined limits at the gateway, fine-grained limits in the app.

Algorithm Deep Dives

Understanding how each algorithm works helps you configure existing tools correctly and debug rate limiting issues. You rarely need to implement these from scratch — but you do need to understand their behavior.

Token Bucket

Token bucket has three parameters: **capacity** (burst size), **refill rate** (sustained throughput), and **cost per request** (usually 1, but can vary by operation).

The algorithm: on each request, calculate tokens accumulated since last check (elapsed time × refill rate), cap at bucket capacity, then check if enough tokens exist. If yes, subtract and allow. If no, reject and calculate retry time.

Python

```python
# Token bucket pseudocode
def check_request(bucket, cost):
    elapsed = now() - bucket.last_update
    bucket.tokens = min(bucket.tokens + elapsed * refill_rate, capacity)
    bucket.last_update = now()

    if bucket.tokens >= cost:
        bucket.tokens -= cost
        return "ALLOW"
    else:
        retry_after = (cost - bucket.tokens) / refill_rate
        return ("REJECT", retry_after)
```

Token bucket core logic.

Here's how traffic patterns play out with a 10-token bucket refilling at 1 token per second:

Traffic Pattern	What Happens
Steady 1 RPS	Every request allowed — token regenerates before next
Burst of 10	All 10 served instantly, then 1 RPS until refilled
Sustained 2 RPS	First ~15 allowed (burst + refill), then 50% rejected
Variable cost (GET=1, POST=5)	Heavy operations consume quota faster

Token bucket behavior under different traffic patterns.

For distributed systems, you need atomic operations. The standard pattern is a Redis Lua script that reads state, calculates refill, checks tokens, and updates — all in one atomic operation. Without atomicity, concurrent requests can race past your limits.

> ✓ SUCCESS
>
> For a complete TypeScript implementation with both in-memory and Redis-backed variants, see the token bucket gist <https://gist.github.com/webstackdev/011a4ac858a9c379b95181798e397499> .

Leaky Bucket

Leaky bucket inverts the model: instead of controlling **input** rate, it controls **output** rate. Requests enter a queue (the bucket) and are processed at a constant rate. If the queue fills, new requests overflow.

Python

```python
# Leaky bucket pseudocode
def check_request(bucket):
    elapsed = now() - bucket.last_leak
    leaked = elapsed * leak_rate
```

```
        bucket.level = max(0, bucket.level - leaked)
        bucket.last_leak = now()

        if bucket.level < bucket.size:
            bucket.level += 1
            wait_time = bucket.level / leak_rate
            return ("QUEUE", wait_time)
        else:
            return "REJECT"
```

Leaky bucket core logic.

The key difference: token bucket serves bursts immediately then makes you wait. Leaky bucket queues everything and serves at constant rate. With a burst of 10 requests:

- **Token bucket (10 tokens, 1/s refill): All 10 served instantly, ~0ms latency each**
- **Leaky bucket (10 queue, 1/s leak): Queued, served over 10 seconds—0ms, 1s, 2s, ... 9s latency**

Use leaky bucket when you need to protect a downstream service that can't handle bursts — like a payment processor with strict per-second limits. The added latency is the tradeoff for guaranteed smooth output.

Characteristic	Token Bucket	Leaky Bucket
Burst handling	Allows bursts up to capacity	Smooths all traffic
Output pattern	Bursty (matches input)	Constant rate
Latency added	None (immediate decision)	Queue wait time
Best for	API rate limiting	Traffic shaping
Rejection behavior	Immediate 429	Queue, then overflow

Token bucket vs leaky bucket comparison.

> ✓ SUCCESS
>
> For a complete TypeScript implementation including a queue-based processor, see the leaky bucket gist <https://gist.github.com/webstackdev/735583b68306d4a553544a43f0eadf60> .

Sliding Window

Sliding window counter solves the fixed-window boundary burst problem with minimal overhead. Track counts for the current and previous windows, then calculate a weighted average based on position within the current window.

Python

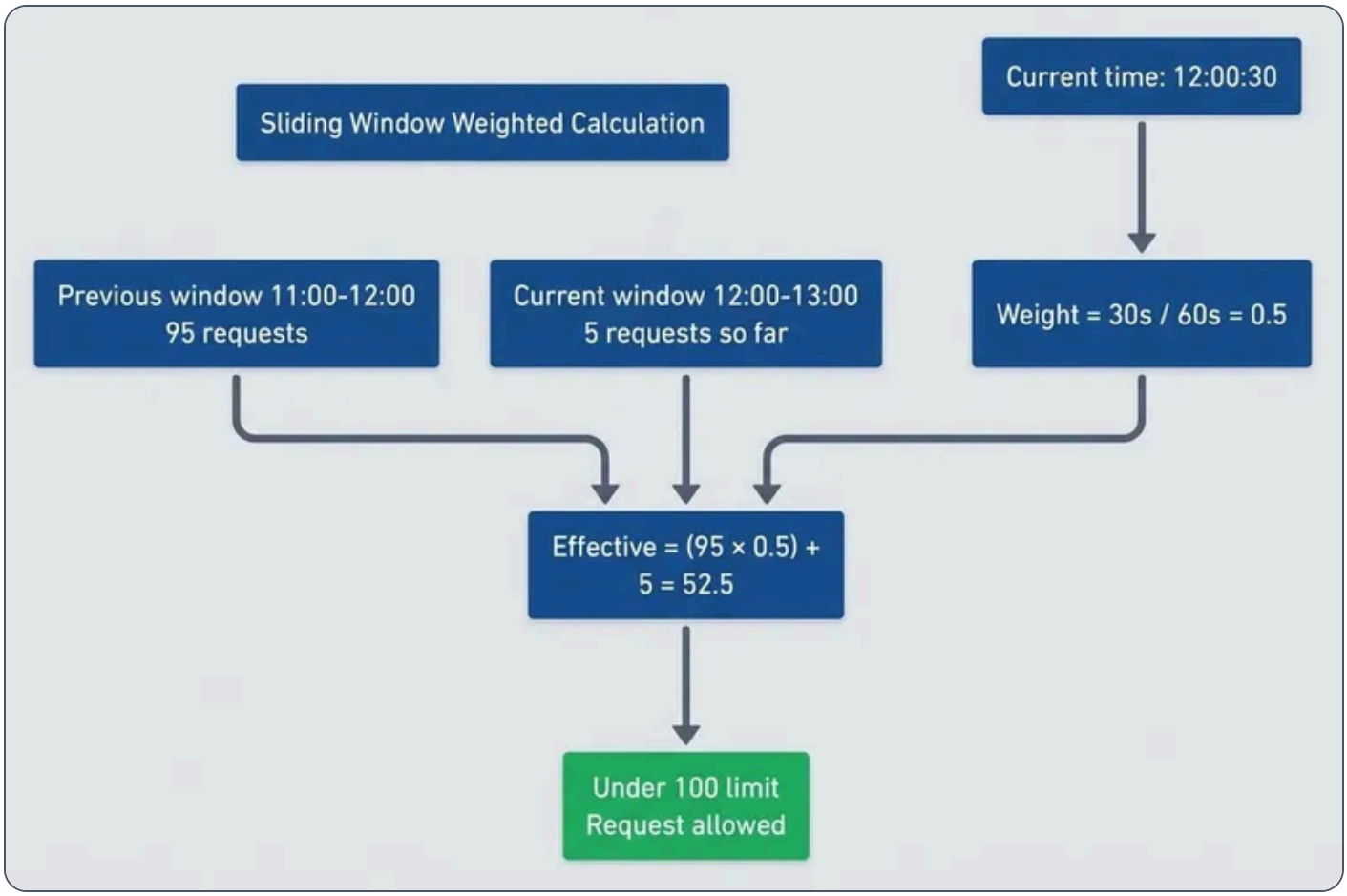
```python
# Sliding window counter pseudocode
def check_request(state):
    current_window = floor(now() / window_size) * window_size
    elapsed = now() - current_window
    weight = elapsed / window_size

    # Weighted count from previous + current window
    effective_count = (state.prev_count * (1 - weight)) + state.current_count

    if effective_count < max_requests:
        state.current_count += 1
        return "ALLOW"
    else:
        retry_after = current_window + window_size - now()
        return ("REJECT", retry_after)
```

Sliding window counter core logic.

At the 30-second mark of a 60-second window, you count 50% of the previous window plus 100% of the current window. This prevents the boundary problem where clients can make 2x the limit by timing requests around window boundaries.



Sliding window weighted calculation prevents boundary exploitation.

> ✓ SUCCESS
>
> For a complete TypeScript implementation with Redis support, see the sliding window gist <https://gist.github.com/webstackdev/2ad5ce11b38b097a261ec3b8e60639c0> .

Client Identification

The rate limiting key — how you identify who's making requests — is as important as the algorithm. Choose wrong, and you'll either punish legitimate users or fail to stop abuse.

#	Strategy	Best For	Watch Out For
1	IP Address	Anonymous, anti-bot	NAT, proxies, shared IPs
2	API Key	B2B, developer APIs	Key sharing, stolen keys
3	User ID	Consumer apps	Auth overhead
4	IP + Endpoint	Mixed anonymous/auth	Configuration complexity
5	User + Action	Fine-grained control	Key explosion

Rate limit key strategies.

1. **IP address**
Requires no authentication and blocks distributed attacks, but corporate NAT can share one IP across thousands of users. I've seen legitimate customers hit rate limits because 500 employees shared a single public IP — each got 0.2 requests per minute.

2. **API key**
The standard for B2B APIs. Per-customer limits, usage tracking, revocation — all straightforward. The risk is key sharing: customers distributing their key to multiple applications, or keys getting stolen and abused.

3. **User ID**
Gives you true per-user limits that work across IPs and devices. The tradeoff is authentication overhead and the fact that it doesn't help with anonymous endpoints.

4. **Composite keys (user + action, IP + endpoint)**
Gives you fine-grained control at the cost of complexity and key explosion. Useful when different operations need different limits.

Extracting Client Identity

For IP extraction, don't blindly trust `X-Forwarded-For` —clients can set it to anything. Only extract from forwarded headers after validating the request came through your trusted proxy infrastructure.

get-client-ip.ts

```
import { APIGatewayRequestAuthorizerEvent, APIGatewayAuthorizerResult } from 'aws-lambda';

// Your original logic integrated here
function getClientIP(headers: Record<string, string | undefined>, remoteAddr: string, trustedProxies: string[]): string {
  if (!trustedProxies.includes(remoteAddr)) return remoteAddr;

  const checkHeaders = ['cf-connecting-ip', 'true-client-ip', 'x-real-ip', 'x-forwarded-for'];
  for (const header of checkHeaders) {
    const value = headers[header];
    if (value) return value.split(',')[0].trim();
  }
  return remoteAddr;
}

export const handler = async (event: APIGatewayRequestAuthorizerEvent): Promise<APIGatewayAuthorizerResult> => {
  const trustedProxies = ['1.2.3.4']; // Replace with your actual proxy IPs
  const clientIP = getClientIP(event.headers || {}, event.requestContext.identity.sourceIp, trustedProxies);

  // In production, check Redis/DynamoDB for rate limit state here
  const isAllowed = true;

  // Standard AWS authorizer policy document - required format for API Gateway
  return {
    principalId: clientIP,
    policyDocument: {
      Version: '2012-10-17',
      Statement: [{
        Action: 'execute-api:Invoke',
        Effect: isAllowed ? 'Allow' : 'Deny',
        Resource: event.methodArn,
```

```
        }],
      },
    };
  };
```

Safe IP extraction from forwarded headers for AWS CloudFront.

The header you trust depends on your infrastructure. Cloudflare sets `CF-Connecting-IP`, Akamai uses `True-Client-IP`, and nginx typically sets `X-Real-IP`. The generic `X-Forwarded-For` contains a comma-separated chain of IPs — the leftmost is the original client, but any proxy in the chain can append values. AWS CloudFront and Application Load Balancer both populate `X-Forwarded-For`, with CloudFront also offering `CloudFront-Viewer-Address` for the viewer's IP and port. For Lambda@Edge or Lambda behind API Gateway, the source IP is available in the request context without needing headers at all.

The safest pattern: configure your edge proxy to set a custom header (like `X-Client-IP`) that overwrites any client-supplied value, then trust only that header in downstream services.

> **⚠ WARNING**
>
> Header spoofing is trivial: `curl -H "X-Forwarded-For: 1.2.3.4" your-api.com`. If your rate limiter trusts that header without validation, attackers can rotate through fake IPs indefinitely. Always validate that requests actually came through your proxy before trusting forwarded headers.

Layered Limiting

Production systems often combine multiple strategies: global IP limits as a DDoS (Distributed Denial of Service) backstop, API key limits for quota enforcement, and user + action limits for abuse prevention. All must pass for a request to proceed.

layered-limits.ts

```
async function checkRateLimits(req: Request): Promise<boolean> {
  const clientIP = getClientIP(req, trustedProxies);
  const apiKey = req.headers['x-api-key'];
```

```
    const userId = req.user?.id;
    const action = `${req.method}:${req.path}`;

    // Layer 1: Global IP limit (anti-DDoS)
    const ipAllowed = await ipLimiter.check(`ip:${clientIP}`);
    if (!ipAllowed) return false;

    // Layer 2: API key quota (billing)
    if (apiKey) {
      const keyAllowed = await keyLimiter.check(`key:${apiKey}`);
      if (!keyAllowed) return false;
    }

    // Layer 3: User + action (abuse prevention)
    if (userId) {
      const actionAllowed = await actionLimiter.check(`user:${userId}:${action}`);
      if (!actionAllowed) return false;
    }

    return true;
  }
```

Layered rate limiting with multiple strategies.

This is an application-level implementation — useful for fine-grained control, but it comes with operational complexity. The `ipLimiter`, `keyLimiter`, and `actionLimiter` shown here are stubs; a real implementation needs shared state across all application instances, typically Redis with atomic Lua scripts. Without shared state, each pod maintains its own counters, meaning a client hitting different pods effectively multiplies their rate limit by your replica count.

Sticky sessions (routing the same client to the same pod) seem like a workaround, but they introduce their own problems: uneven load distribution, session affinity breaking during deployments, and the need for session-aware load balancers. Redis-backed rate limiting is the standard solution for horizontally scaled services.

That said, consider whether you need application-level rate limiting at all. Cloud platforms offer managed alternatives that handle the distributed state problem for you. AWS API Gateway's usage plans provide per-API-key throttling with built-in token bucket. Lambda Authorizers can implement custom logic for user-based limits. Kong, Envoy, and other gateways support sophisticated rate limiting plugins with Redis backends. Moving rate limiting to the gateway layer reduces application complexity and ensures limits are enforced before requests consume compute resources.

HTTP Response Design

Good rate limiting is invisible to clients until they approach the limit. The key: always return rate limit headers on *every* response, not just 429s. This lets well-behaved clients self-throttle before hitting limits.

Standard Headers

The IETF draft (draft-ietf-httpapi-ratelimit-headers) defines three standard headers:

Header	Meaning	Example
`RateLimit-Limit`	Maximum requests allowed	`100`
`RateLimit-Remaining`	Requests left in window	`47`
`RateLimit-Reset`	Seconds until reset	`30`

IETF rate limit headers.

Many APIs still use the legacy `X-RateLimit-*` prefix. Support both until the standard is finalized.

For 429 responses, always include `Retry-After` (RFC 7231) telling the client when to retry. Use seconds rather than HTTP-date format — it's simpler for clients to parse.

Response Codes

429 Too Many Requests
Client exceeded their rate limit. Include `Retry-After` and rate limit headers.

503 Service Unavailable
System-wide overload, not per-client limiting. Use sparingly.

200 with Warning
Optional. Add `Warning: 299 - "Rate limit 80% consumed"` to help clients self-regulate.

The distinction between `429` and `503` matters: `429` tells clients "you specifically are sending too many requests," while `503` signals "the entire system is overloaded." Don't use `503` for per-client rate limiting — it misleads clients into thinking the service is down rather than that they need to back off.

Example Responses

rate-limit-responses.ts

```
// 429: Client-specific rate limit exceeded
res.status(429)
   .set({ 'Retry-After': '30', 'RateLimit-Remaining': '0' })
   .json({ error: 'rate_limit_exceeded', retryAfter: 30 });

// 503: System-wide overload (use sparingly)
res.status(503)
   .set({ 'Retry-After': '60' })
   .json({ error: 'service_overloaded', message: 'System under heavy load' });
```

429 vs 503 response examples.

Variable Cost

Remember the “cost per request” parameter in token bucket? Not all requests are equal. A simple GET costs less than a complex search or bulk export. Charge more tokens for expensive operations:

variable-cost.ts

```
function getRequestCost(req: Request): number {
  if (req.path.includes('/search') || req.path.includes('/export')) return 10;
  if (req.method === 'POST' || req.method === 'PUT') return 3;
  return 1;
}
```

Variable request costs.

✓ SUCCESS

Always include rate limit headers on successful responses, not just `429` s. This lets well-behaved clients monitor their quota and self-throttle before hitting limits.

Testing Your Rate Limiter

Rate limiters need to be tested under realistic conditions — not just unit tests, but concurrent load tests that stress the actual distributed implementation. Tools like k6, Locust, or Grafana's k6 Cloud can generate the concurrent traffic patterns you need to verify your limiter behaves correctly under pressure.

Key Test Cases

Burst handling Send requests up to capacity simultaneously. Exactly N should succeed.
Refill behavior Drain the bucket, wait for refill period, verify new requests succeed.
Concurrent access Multiple processes hitting the same key. Redis Lua scripts must be atomic — race conditions here mean either over-allowing or unfair rejection.
Retry-After accuracy The header value should match when requests actually start succeeding.

concurrent-test.ts

```
it('handles concurrent requests atomically', async () => {
  const results = await Promise.all(
    Array(15).fill(null).map(() => bucket.consume('test-key'))
  );

  const allowed = results.filter(r => r.allowed).length;
  expect(allowed).toBe(10); // Exactly capacity, no race conditions
});
```

Testing atomic concurrent access.

What to Monitor

Rejection rate High sustained rejection for legitimate clients means misconfiguration. High during an attack means it's working.	
Rate limiter latency Redis calls should be under 5ms P99. If slower, check network or Redis load.	
Redis errors Any failures here mean rate limiting isn't happening. Alert immediately.	
Top rate-limited clients Identify who's hitting limits — abuse or legitimate growth?	

INFO

A high rejection rate isn't always a problem — it might mean the rate limiter is working as intended during an attack. Monitor both aggregate rejection rate and per-client patterns.

Failure Handling

What happens when Redis dies? This isn't hypothetical — network partitions, memory exhaustion, and maintenance windows all cause Redis outages. Your rate limiter needs a failure policy decided in advance.

Fail open allows all requests when the rate limiter can't be reached. This preserves availability but removes protection. One compromised API key during an outage can generate unbounded requests.	

Fail closed rejects all requests when state is unavailable. This maintains protection but sacrifices availability. A Redis blip becomes a full outage.
Fail with local fallback uses in-memory rate limiting per instance when Redis is unavailable. It's not perfectly accurate — each pod limits independently — but it provides some protection while maintaining availability.

Most production systems choose fail open with aggressive alerting. The reasoning: Redis outages are rare and short-lived, while blocking all traffic during an outage compounds the problem. But document your choice and make sure operations knows the implications.

fail-open-with-fallback.ts

```
async function checkRateLimit(key: string): Promise<boolean> {
  try {
    return await redisLimiter.check(key);
  } catch (error) {
    metrics.increment('rate_limit.redis_failure');
    // Fail open: allow request but log for alerting
    return true;
  }
}
```

Fail-open rate limiting with error tracking.

Conclusion

Rate limiting is traffic shaping, not just request blocking. The best rate limiters are invisible to normal users — they only activate during abuse or overload.

The key insights:

- **Layer appropriately**
  Edge for DDoS, gateway for API quotas, application for business logic

- **Choose algorithms by burst tolerance**
  Token bucket allows bursts, leaky bucket smooths traffic, sliding window prevents boundary exploits
- **Identify clients carefully**
  IP for anonymous endpoints, API key for B2B, user ID for authenticated actions
- **Communicate clearly**
  Always return `Retry-After` and rate limit headers, even on successful responses
- **Plan for failures**
  Decide your failure policy before Redis goes down, not during the incident

The goal isn't to reject requests — it's to shape traffic so rejection becomes rare. Design for legitimate bursts, communicate limits clearly, and monitor rejection rates. Rate limiting done right protects your service without punishing your users.

> ✓ SUCCESS
>
> Well-designed rate limits with clear communication let clients self-regulate before hitting limits. That's the real goal.

https://www.webstackbuilders.com/articles/rate-limiting-token-bucket-leaky-bucket-implementation